

I. Performance

Subset	Resolved Count	Resolved ID(s)
verified-40	16	"django__django-11532", "django__django-11880", "django__django-13089", "django__django-13363", "django__django-14017", "django__django-14855", "django__django-15103", "django__django-15315", "django__django-15368", "django__django-15987", "django__django-16136", "django__django-16661", "pydata__xarray-6461", "pytest-dev__pytest-5809", "pytest-dev__pytest-7982", "sphinx-doc__sphinx-8721"
verified-45-extra	2	"django__django-16642", "pydata__xarray-6744"
verified-50-extra	1	"scikit-learn__scikit-learn-13135"

Separate preds and eval files have been created for each subset and are defined below:

1. Verified-40
 - a. Eval file: deliverables_midterm/gemini__gemini-2.5-flash.verified-40-eval.json
 - b. Preds file: deliverables_midterm/generation/verified-40.preds.json
2. Verified-45-extra
 - a. Eval file:
deliverables_midterm/gemini__gemini-2.5-flash.verified-45-extra-eval.json
 - b. Preds file: deliverables_midterm/generation/verified-45-Extra.preds.json
3. Verified-50-extra
 - a. Eval file:
deliverables_midterm/gemini__gemini-2.5-flash.verified-50-extra-eval.json
 - b. Preds file: deliverables_midterm/generation/verified-50-Extra.preds.json

Design - Write Tool:

I. Objective

Implement a lightweight `write` tool for `mini-swe-agent` that allows the agent to create or overwrite files

safely while keeping the framework simple and reliable.

Key Requirements

- Accept a target `file\_path` and `content` to write.
- Ensure directories exist or fail gracefully.
- Overwrite files by default; optionally support append in future iterations.
- Return clear status messages for success or failure.
- Avoid security issues: restrict writes to the workspace root and block traversal outside it.

II. Proposed Interface

@tool

```
def write_file(file_path: str, content: str) -> str:
```

```
    """Write content to a file inside the workspace."""
```

Implementation Steps

1. ****Normalize Path****

- Convert `file\_path` to an absolute path relative to the agent's workspace root.
- Validate it remains inside the workspace; reject if not.

2. ****Prepare Directories****

- Create missing parent directories with `mkdir(parents=True, exist\_ok=True)` so the agent can write nested files without manual setup.

3. ****Write Content****

- Open the file in write mode (`"w"`). This keeps the first version minimal; append mode can be added later.
- Write the `content` string as-is.

4. ****Handle Errors****

- Catch `OSError`/`IOError` and return the error message in a structured string.
- Include the normalized path in messages for debugging.

5. ****Result Format****

- On success: `"write\_file: wrote <bytes> bytes to <path>."`
- On failure: `"write\_file error: <exception message>."`

III. Interactions/Implementation

sequenceDiagram

participant Agent

participant ToolRegistry as Tool Registry

participant WriteTool as Write Tool

participant FileSystem as File System

Agent->>ToolRegistry: request write_file(file_path, content)

ToolRegistry->>WriteTool: invoke write_file(...)

WriteTool->>WriteTool: normalize and validate path

WriteTool->>FileSystem: create directories and write content
FileSystem-->>WriteTool: success or error
WriteTool-->>ToolRegistry: status message
ToolRegistry-->>Agent: return response

IV. Testing Strategy

- Write unit tests covering:
 - Writing a new file.
 - Overwriting an existing file.
 - Attempting to write outside the workspace (should fail).
 - Handling invalid paths or permission issues.
- Add integration tests via agent simulation if time allows.

Future Enhancements

- Optional `mode` parameter for append vs overwrite.
- Support for binary data via base64 input.
- Combine with a `replace` tool to minimize full rewrites.

Design - Read Tool

I. Objective

Read only the useful lines from a file to cut tokens, speed up runs, and keep the agent focused. Keep the tool small, safe, and line-centric.

- Support targeted reads:
 - lines: exact line numbers
 - range: inclusive start–end
 - head: first N lines
 - tail: last N lines
 - around: window around a line (radius)
- Lines are 1-based. One selector per call.
- Hard cap output via max_chars.
- Deny path traversal; reject binary files; normalize \n.

II. Proposed Interface

Name: read

Args:

...

```
{
  "path": "src/module/file.py",
  "selector": {
    "lines": [12, 18, 24],
    "range": {"start": 50, "end": 80},
    "head": 200,
    "tail": 200,
    "around": {"line": 320, "radius": 20}
  },
  "strip_comments": false,
  "max_chars": 20000
}
```

```
...
Rules: Provide exactly one of lines / range / head / tail / around.
Result:
...
```

```
{
  "path": "src/module/file.py",
  "encoding": "utf-8",
  "start_line": 301,
  "line_count": 41,
  "truncated": false,
  "content": "..."
}
...
```

```
Pseudocode:
...
resolve repo root -> resolve(path) -> deny traversal
reject binary
read text -> splitlines()
pick slice by selector
optional strip comments (light heuristics)
join; enforce max_chars; return payload
...
```

III. Rationale

- Line-based access matches diffs and patches.
- Token reduction reduces 429s and cost.
- around mirrors how developers inspect code near stack traces or grep hits.
- Avoid AST parsing to stay minimal; push advanced logic to future tools.

Security & Safety

- Resolve real path; forbid escaping repo root (... symlinks).
- Reject likely binary (NUL byte check).
- Enforce max_chars.
- Treat unknown encodings as UTF-8 with errors="replace".

Edge Cases

- Out-of-bounds lines $\Rightarrow$ clamp to file length and note it.
- Empty result (e.g., tail:0) is allowed.
- CRLF normalized to `\n`.
- strip_comments: best-effort for .py and // languages; no-ops elsewhere.

IV. Interactions/Implementation

- src/minisweagent/tools/read.py — new tool implementation.
- src/minisweagent/tools/__init__.py — export symbol.
- src/minisweagent/config/tools.yaml — bind tool name $\rightarrow$ function path.

V. Testing

- Unit: each selector, clamping, truncation, traversal block, binary reject, unicode.
- Golden: run against a real repo file and verify line numbers.
- Smoke: call via agent once and verify output shape + char cap.